

SCP Research Record: Congestion Signals, Externalities, and the Dynamics of Pacing

Skai Uijing
LTTit Research

Research record, October 6, 2026

Abstract—SCP began as a reliable transport for LTTit’s distributed embedded runtime. Poor behavior under random loss prompted experiments with a nonlinear loss-and-delay controller, followed by deeper questions about fairness, responsibility for congestion, and the interaction between window growth and packet scheduling. This paper records that progression rather than presenting a single finished controller. We describe the implemented PROB law and its documented fairness failures; explain why congestion severity, marginal responsibility, and economic externality are different quantities; and summarize the subsequent explicit-feedback and receiver-control investigations. The most developed mathematical result is an exact analysis of a specified per-ACK append scheduler. Under fixed RTT, one ACK per packet, and unbounded slow start, harmonic blocks give a growth exponent of $\min(g/2, \log 2)$ and a transition at $g = 2 \log 2$. These results concern that scheduler, not all pacers or the current SCP implementation. Historical measurements, deterministic simulations, finite rational checks, and conjectures are identified separately. The retained work supplies models and testable boundaries; multi-flow stability, implementation correspondence, and deployment benefits remain open.

Index Terms—congestion control, pacing, ACK feedback, externality, reliable transport, research history

I. FROM AN EMBEDDED TRANSPORT TO RESEARCH QUESTIONS

SCP provides reliability, flow control, and congestion control between CCNET routing and CCRPC operations in LTTit. The March 7 development post describes a Reno-based, ACK-driven implementation and a long bidirectional file-transfer experiment. The June 14 post asks why random loss should always force the same response as congestion. That question led to a Bayesian-inspired engineering score, nonlinear window updates, and the PROB implementation. The posts record observations and evolving explanations; their front-matter dates do not establish when every later edit was made.

The subsequent research did not stop at finding a more aggressive increase rule. Poor competition results led to questions about who causes congestion and who should pay. September notes distinguish finite-horizon actionability, counterfactual harm to competing flows, explicit exchange of resource information, and receiver-issued credits. A separate investigation was motivated by an observation that changing SCP from burst transmission to pacing could slow transfer without a comparable change in useful-byte fraction. This motivated an analysis of the physical timing of ACK feedback, rather than another utility function chosen in advance.

TABLE I
PRESERVED RESEARCH PROGRESSION. RECORDS ARE NOT EQUIVALENT TO DEPLOYMENT EVIDENCE.

Record	Question and current status
March–June posts	Reliable Reno transport; random-loss performance; Bayesian-inspired score experiments.
August PROB	Implemented nonlinear controller; historical throughput and fairness reports, including failures.
September 9	Code/formula errata; pacing-aware and acknowledged-service candidates, with simulation limits.
September 16–17	Externality counterexample, ideal price feedback, explicit information and receiver-control literature.
September 25	Exact per-ACK append model, general-gain proof, finite rational verification; CA conjecture retained.
September 29–October	BBR gain comparison and implementation questions; no established equivalence to BBR or Linux.

The inspected standalone SCP source is based on revision bf9d887; LTTit’s embedded SCP is a different snapshot and still contains Reno-style control. Claims about the standalone PROB experiment must not be attributed automatically to the integrated firmware. This paper consolidates existing evidence and performs no new network benchmark.

II. IMPLEMENTED PROB: WHAT THE SIGNAL MEANS

Let W be the byte window, M the MSS, b newly acknowledged bytes, and ℓ a smoothed loss estimate. The code’s RTT hook forms an effective sample

$$r_{\text{eff}} = r_{\text{raw}} + \tilde{\ell} (200 \text{ ms}). \quad (1)$$

After smoothing, the score uses $d = \max(0, \text{srtt} - r_{\text{base}})$ and a sigmoid

$$q = \frac{1}{1 + \exp[-(\gamma + \beta d)]}. \quad (2)$$

Configured anchors place $q = .2$ at $d_{\min}$ and $q = .8$ at $d_{\min} + s$, so $\beta = 2 \log 4/s$ and $\gamma = -\log 4 - \beta d_{\min}$. Fixed-point evaluation and an exponential moving average implement the score. It is not an empirically calibrated posterior probability. Adding separate log-likelihood terms would require a specified statistical model, including conditional-independence assumptions; correlated delay and loss do not establish that model by themselves.

TABLE II
SELECTED AUGUST 19 README REPORTS. RANDOM-LOSS PERCENTAGE
IS A TEST CONDITION; THESE ARE NOT NEW RUNS.

Flows	Loss	Total Mbit/s	Jain index
One PROB	0%	4.686	—
One AIMD	0%	4.746	—
Two PROB	0%	4.735	.641
PROB + AIMD	0%	3.421	.504
Two PROB	10%	2.044	.999887

The controller has increase and decrease phases, switching at $q \geq .6$ and $q \leq .4$, respectively. In ideal real-valued notation, the corresponding changes are

$$a(q) = 1 - [q/(9/13)]_{[0,1]}^2, \quad (3)$$

$$\Delta W_{\text{inc}} = \max\{1, a(q)Mb / \max(W, 1)\}, \quad (4)$$

$$h(q, \bar{q}) = [(\max(q, \bar{q}) - 4/13)/(1 - 4/13)]_{[0,1]}^2, \quad (5)$$

$$\Delta W_{\text{dec}} = -.3h(q, \bar{q})(W - M) \min\{1, b/W\}. \quad (6)$$

The implementation then clips the window and converts to an integer. The configured MSS is 1,430 bytes. Loss and timeout callbacks in PROB do not directly impose Reno's window cut, but transport retransmission and recovery remain active.

The September 9 erratum matters mathematically. An older formula omitted the ACK-ratio cap in (6); a cumulative ACK can exceed a recently reduced window. RTT sampling and new-byte acknowledgement are also separate events in the code. A proof about equal-signal, equal-byte, per-ACK maps is not a proof of equal rates in wall-clock time. For example, ignoring clipping in the increase branch,

$$\frac{dW}{dA} = \frac{a(q)M}{W}, \quad \frac{dA}{dt} \simeq \frac{W}{R} \implies \frac{dW}{dt} \simeq \frac{a(q)M}{R}. \quad (7)$$

Two windows with the same q, R then have equal first-order slopes; their absolute difference need not contract. Heterogeneous ACK rates and switching times require their own analysis.

A. Historical Results, Including Failure

The August 19 README reports a five-Mbit/s configuration, 50-ms one-way delay, a 1,000-packet queue, 100-MiB single-flow transfers, and 300-second competition tests. Table II preserves selected results from that record. Its raw per-run logs are not bound to the inspected revision, so these values are historical reports, not independently reproduced measurements. The earlier six-page PROB PDF contains a different result set and must not be pooled with this table.

In the zero-random-loss mixed test, PROB obtained only about 0.36% of delivered bytes. Two PROB flows had a reported throughput ratio of about 6.95 without random loss, despite near-equal sharing in the 10% case. These outcomes motivated the attribution investigation. They do not support a general fairness or global-stability claim, nor do the favorable high-loss observations establish superiority over modern transport baselines.

III. FROM CONGESTION ATTRIBUTION TO EXTERNAL COST

The initial economic intuition was that a flow causing congestion should bear its cost. September's refinement asks what quantity can actually be identified and controlled. Occupancy share, a probability that the network is congested, the effect of changing one's own rate, and harm to other users are not interchangeable.

A. A Static Distinction

For total load $X = \sum_i x_i$ and per-unit congestion cost $d(X)$, private cost is $h_i = x_i d(X)$ and total cost is $H = X d(X)$. Holding other rates fixed,

$$\frac{\partial H}{\partial x_i} = d + X d', \quad \frac{\partial h_i}{\partial x_i} = d + x_i d', \quad (8)$$

so marginal external cost is

$$\tau_i = (X - x_i) d'(X). \quad (9)$$

A measured own-rate sensitivity $x_i d'$ therefore still needs other-load information to recover τ_i . A budget-balanced proportional charge $x_i d(X)$ is not automatically an externality tax. Conversely, multiplying the marginal tax by x_i does not generally give a total payment with the desired derivative. With other rates fixed, one such integrated payment is $(X - x_i)[d(X) - d(X - x_i)]$.

The draft defines a finite-horizon counterpart using the other flows' net payoffs P_j and a perturbation ϵ of sender i :

$$E_i(T) = - \frac{d}{d\epsilon} \sum_{j \neq i} P_j(T; \epsilon) \Big|_{\epsilon=0}. \quad (10)$$

Effective delivered work, cost weights, horizon, controller policy, and actual actuator perturbation must all be specified. It is not legitimate to hold competing rates fixed while claiming to have measured their closed-loop response. Different unobserved utility weights can produce the same RTT trace but different E_i ; local timing alone cannot universally identify that economic quantity.

B. Why Queue Recovery Can Hide Harm

The September 16 notes construct a transparent PI example. Let z be a queue deviation, $u = \epsilon$ a constant added input, and v a competing controller's response:

$$\dot{z} = u + v, \quad v = -2z/\theta - b/\theta^2, \quad \dot{b} = z, \quad (11)$$

with zero initial deviations. Direct solution gives

$$z(t) = \epsilon t e^{-t/\theta}, \quad (12)$$

$$v(t) = \epsilon[(1 - t/\theta)e^{-t/\theta} - 1]. \quad (13)$$

The queue deviation returns to zero while the competitor's rate change tends to $-\epsilon$. Thus a terminal queue signal can disappear because someone else yielded. This does not make every queue statistic vanish: integrated extra delay through capacity C is

$$\frac{\epsilon \theta^2}{C} \left[1 - (1 + T/\theta)e^{-T/\theta} \right]. \quad (14)$$

The accompanying RK4 check tests this ideal ODE, not TCP or SCP. The example assumes an interior queue, small perturbations, and no feedback delay or actuator saturation. Its role is to refute an overly broad identification claim, not to prove a universal impossibility theorem for all signals.

C. What a Dynamic Derivative Would Require

The economic derivation draft also makes the missing information explicit. For a finite-horizon state model $z_{k+1} = F(z_k, \theta, \xi_k)$, a fixed initial state and a smooth fixed event branch give the sensitivity recursion

$$S_{i,0} = 0, \quad S_{i,k+1} = A_k S_{i,k} + B_{i,k}, \quad (15)$$

where $S_{i,k} = \partial z_k / \partial \theta_i$, $A_k = \partial F / \partial z$, and $B_{i,k} = \partial F / \partial \theta_i$. Other users' payoff derivatives then involve both their direct parameter dependence and $S_{i,k}$. An adjoint can reorganize the calculation, but cannot supply unknown dynamics or unobserved utility weights. Packet drops, switching branches, and saturation also require care before differentiating a trajectory.

This formulation clarifies why a requested window perturbation is insufficient evidence. An application-limited sender, receiver window, or already frozen pacing batch can prevent that request from changing actual offered traffic. The proposed experimental record therefore includes the requested action, actual new and retransmitted bytes, pending credit, and raw timing. No production estimator of these finite-horizon externalities has been implemented or validated.

D. An Ideal Economic Example, Not a Packet Controller

One solvable draft example normalizes capacity to 1, assumes $0 < X < 1$, and chooses $d(X) = X/(1-X)$ with payoff

$$P_i = \alpha_i \log x_i - \beta x_i d(X), \quad \alpha_i, \beta > 0. \quad (16)$$

Total welfare is $\mathcal{W} = \sum_i \alpha_i \log x_i - \beta X^2/(1-X)$. Correcting private gradients by the external cost gives

$$E_i = \frac{\beta(X - x_i)}{(1-X)^2}, \quad (17)$$

$$\partial_i \mathcal{W} = \frac{\alpha_i}{x_i} - \pi(X), \quad \pi(X) = \frac{\beta X(2-X)}{(1-X)^2}. \quad (18)$$

Its Hessian is

$$\nabla^2 \mathcal{W} = -\text{diag}(\alpha_i/x_i^2) - \frac{2\beta}{(1-X)^3} \mathbf{1}\mathbf{1}^T \prec 0. \quad (19)$$

Consequently there is a unique interior welfare optimum. If $A = \sum_i \alpha_i$, it satisfies $x_i^* = \alpha_i X^*/A$ and

$$A(1-X^*)^2 = \beta(X^*)^2(2-X^*). \quad (20)$$

The left side decreases and the right increases on $(0, 1)$, so the aggregate solution is unique. The example shows how a fully specified cost yields a tractable optimization problem. It does not identify that cost from SCP observations or prove stability with delayed feedback. Its assumptions differ from the capacity-constrained price system below.

E. Explicit Information and Receiver Control

The investigation then moved from inference alone to explicit reports of bytes, active flows, resource groups, and common time windows. For a known single resource of capacity C , one ideal Kelly-type system [1] is

$$\dot{x}_i = \kappa_i(\alpha_i - p x_i), \quad \dot{p} = \gamma[\sum_i x_i - C]_p^+, \quad (21)$$

where projection prevents a negative price. Its equilibrium is $x_i^* = \alpha_i C / \sum_j \alpha_j$, $p^* = \sum_j \alpha_j / C$. The recorded Lyapunov function is

$$V = \sum_i \frac{x_i - x_i^* - x_i^* \log(x_i/x_i^*)}{\kappa_i} + \frac{(p - p^*)^2}{2\gamma}. \quad (22)$$

Substitution yields $\dot{V} \leq -\sum_i \alpha_i (x_i - x_i^*)^2 / (x_i x_i^*)$. This supports the ideal no-delay model with complete resource membership; it does not prove stability of heterogeneous TCP controllers, token losses, unknown bottlenecks, or instantaneous capacity feasibility.

Receiver tokens and least-served-first scheduling have substantial prior art. pHost explicitly discusses fairness based on service counts [2]; ExpressPass enforces credits along network paths [3]; SIRD separates receiver, sender, and network constraints [4]. The local September 17 survey therefore narrowed the candidate questions to accounting under loss, correct resource scope, limited information, and interaction with heterogeneous senders. A token granting a bounded number of bytes is different from a permanent instruction to increase rate. Local process flow lists are not a census of all users of a physical bottleneck. No completed explicit-feedback or receiver-control deployment is claimed here.

The later explicit-information note also considers a managed resource that retains heterogeneous sender controllers. It first sets a total admission budget, then allocates feasible shares such as $b_i = \min(d_i, w_i \nu)$ under demand limits d_i . A price, a byte grant, an ECN mark, and a hard shaper have different actuator semantics. Marking a fraction of excess traffic does not, by itself, prove that a delayed AIMD sender will adopt exactly the assigned rate. Dropping a packet after the bottleneck cannot recover capacity it already consumed. The ideal Lyapunov argument above does not cover this extension; cooperation, enforcement location, credit expiration, and control-loop interaction remain design questions.

IV. PACING: THE SCHEDULER IS PART OF THE MODEL

Pacing controls when available window credit becomes actual traffic. If ACKs depend on actual send times, unchanged per-ACK window updates need not produce unchanged $W(t)$. The research first used a fixed-RTT batch comparison, then refined it into an explicit reservation model. A batch pacer sampled once each RTT has the Fibonacci recurrence $L_{n+2} = L_{n+1} + L_n$, with $L_0 = 1, L_1 = 2$; immediate sending gives 2^n . A Fibonacci lower bound needs scheduler-specific service conditions. It is not a theorem about every implementation called pacing.

An earlier September 9 acknowledged-service candidate accumulated credit $\Delta\Theta = \sum \text{bytes}/p_{\text{first}}$ and adjusted windows

on that clock. Under actual send rate equal to the configured pacing rate, this clock can track elapsed service time. That equality can fail under a window, application, or scheduler limit. Retained deterministic tests did not show a universal queueing advantage: in a 20-Mbit/s unlimited-FIFO stress experiment, maximum queue waiting was about 1.997 seconds for the candidate versus 1.578 seconds for the burst baseline. This candidate remains separate from the exact scheduler analysis below.

A. Exact Per-ACK Append Scheduler

Normalize time by fixed RTT R and window by MSS M . Assume no loss, unlimited data, one ACK per packet, infinite slow start, no receiver-window limit, and no finite serialization time. Start with one packet sent at $s_0 = 0$, window 1, and tail timestamp $T_0 = 1/g$, for $g \geq 1$.

An ACK releases one in-flight slot and increases the window by one; it therefore authorizes two packet reservations. Reservations append to the queue, retain old timestamps, use the post-ACK window, and preserve a trailing pacing interval. After each reservation event, $F + Q = w$, where F is in-flight and Q is reserved but unsent credit. The exact recursion for ACK $k \geq 1$ is

$$a_k = s_{k-1} + 1, \quad w_k = k + 1, \quad (23)$$

$$b_k = \max(a_k, T_{k-1}), \quad (24)$$

$$s_{2k-1} = b_k, \quad s_{2k} = b_k + \frac{1}{g(k+1)}, \quad (25)$$

$$T_k = b_k + \frac{2}{g(k+1)}. \quad (26)$$

This specifies more than a nominal rate gW/R . Removing the trailing interval, repricing pending packets, or waiting for an entire batch to empty defines a different scheduler.

B. Harmonic Blocks and the Gain Transition

Put $c = 2/g$, $H_k = \sum_{r=1}^k 1/r$, $D(j) = H_{2j} - H_j$, and $B_m = b_{2^m}$.

Proposition 1. *For the scheduler (26) and every $g \geq 1$,*

$$B_0 = 1, \quad B_{m+1} = B_m + \max\{1, cD(2^m)\}, \quad (27)$$

$$b_k = B_m + c(H_k - H_{2^m}), \quad 2^m \leq k < 2^{m+1}. \quad (28)$$

Proof. The initial ACK gives $b_1 = 1$. For $q = 2^m$, its ACK arrives at $B_{m-1} + 1$, and the previous block's tail is $B_{m-1} + cD(q/2)$, giving the block-start formula. For an interior even index $2j > q$, the candidate tail minus its ACK time is

$$\Delta - 1 + c[D(j) - D(q/2)] > 0,$$

where $\Delta = B_m - B_{m-1} \geq 1$. For an interior odd index $2j + 1 > q$, the difference is

$$\Delta - 1 + c \left[D(j) - D(q/2) + \frac{1}{2j+1} - \frac{1}{2j+2} \right] > 0.$$

Here $D(j+1) - D(j) = 1/[(2j+1)(2j+2)] > 0$. Thus all interior events use the queue tail; summing the intervals gives (28) and completes induction. $\square$

TABLE III
SELECTED ARCHIVED APPEND-MODEL CHECKS AT $\tau = 18$. WINDOWS ARE IN MSS UNITS. THE ASYMPTOTIC FORMULA INCLUDES THE GAIN-DEPENDENT PREFACTOR; THESE ARE DETERMINISTIC COMPUTATIONS, NOT TRANSPORT MEASUREMENTS.

Gain	Event-model window	Asymptotic prediction
1.00	9,099	9,099.107
1.25	56,912	56,912.191
1.38	127,909	127,909.544

Only power-of-two ACK indices can add waiting beyond the reserved tail. These are analysis blocks, not RTT batching instructions. A block with $q = 2^m$ contains $2q$ sends. Its first-to-last send span L_m , extra ACK wait I_m , and last-to-next-first gap G_m satisfy

$$L_m/R = cD(q) - 1/(2gq), \quad (29)$$

$$I_m/R = [1 - cD(q)]_+, \quad (30)$$

$$G_m/R = [1 - cD(q)]_+ + 1/(2gq). \quad (31)$$

Send span is not link busy time: this model has no packet serialization duration. Finite-block waiting occurs exactly when $g > 2D(2^m)$. The thresholds begin at $1, 7/6, 533/420$ and increase to $g_* = 2 \log 2$.

For $1 \leq g < g_*$, only finitely many blocks wait. Eventually $b_k = cH_k + \delta$, hence $w(\tau) = A_g e^{g\tau/2} + O(1)$ for a constant $A_g > 0$. For $g \geq g_*$, every block starts at $B_m = m + 1$. Since block counts double, and within-block counts differ by at most a constant factor,

$$\lim_{\tau \rightarrow \infty} \frac{\log w(\tau)}{\tau} = \min\{g/2, \log 2\}. \quad (32)$$

At the critical gain, finite-block waits stay positive but approach zero; above it their normalized limit is $1 - 2 \log 2/g$. At $g = 1$, $b_k = 2H_k - 1$ exactly and $w(\tau) = 2e^{\tau/2 - \gamma_E} + O(1)$. The result explains a particular feedback loop, not optimal throughput on a congested network.

C. Verification and the Unfinished CA Result

The archived September 25 program compares an independent rational SEND/ACK event simulation with the block formulas for 13 rational gains through nine RTTs. All recorded batch and span comparisons agree. These are finite checks accompanying a proof, not a substitute for induction. Five subcritical-gain runs also compare the asymptotic prefactor at 18 RTTs; each recorded error is less than one window unit. They are deterministic model outputs, not packet captures.

When the same model enters congestion avoidance, the queue clock has a derived decomposition involving window size, its logarithm, and accumulated ACK waiting. The stronger proposed asymptotic

$$w(\tau) = \tau - \frac{3}{2} \log \tau + O(1) \quad (33)$$

still depends on an unproved waiting-term estimate. The archive reports numerical support through 3,000 RTTs, but

explicitly retains it as a conjecture. It is not a result of Proposition 1, which assumes infinite slow start.

V. RELATION TO BBR AND ACTUAL IMPLEMENTATIONS

The Google BBR team’s 2018 derivation [5], [6] assumes a continuous unsaturated startup with sufficient congestion window. In RTT-normalized units, let $p(t) = k2^t$ and let the modeled bandwidth sample be

$$\hat{B}(t) = \int_{t-2}^{t-1} p(u) du. \quad (34)$$

Then $p = G\hat{B}$ gives $G = 4\log 2$; the associated flight requirement gives a target cwnd gain $K = 2$. Only if actual $W = K\hat{B}R$ does this imply $p = (G/K)W/R = 2\log 2 W/R$. BBR’s bandwidth-relative gain and our window-relative gain have different denominators. Numerical coincidence is not equivalence of samplers, transient windows, or packet schedules.

The latest blog explores this connection to Linux 6.6 and finite windows. The supported archival conclusion is a comparison of models, not a proof that BBR follows the exact append recursion. Pacing research already covers performance effects and implementation differences [7], [8]. A gain threshold found in this model does not establish a previously unknown reason for every Linux pacing constant.

The current standalone SCP creates a batch of deadlines in `scp_output`; an existing pending batch prevents immediate replanning. Millisecond timer callbacks can release several overdue packets, and retransmissions use a separate path. This differs from per-ACK appending and from a fluid pacer. The required next check is an event-level correspondence: actual SEND and ACK times, window, flight, reserved queue, pacing rate, recovery state, and application/receiver limits. No mathematical prediction should be promoted to an SCP performance result before this correspondence is established.

VI. WHAT HAS BEEN ACHIEVED AND WHAT REMAINS

The work has produced an implemented nonlinear controller, adverse competition observations, corrected implementation formulas, explicit counterexamples to simplistic attribution, ideal economic-feedback models, and a precise scheduler theorem with finite verification. It has not produced

one controller that simultaneously inherits all those results. In particular, the no-delay price-model proof cannot certify PROB, and the lossless single-flow pacing theorem cannot certify fairness.

The next empirical study should preserve those separations. First validate the actual scheduling trace against an explicit event model; then vary ACK ratio, timer granularity, finite link service, and recovery one at a time. Attribution experiments need reset or paired initial states and must record actual transmitted perturbations rather than requested window changes. Receiver feedback needs bounded credit accounting, loss/duplicate handling, and complete definitions of the resource group. Multi-flow tests must retain startup offsets, queue measurements, and per-flow outcomes, including failures.

This record does not claim a new general congestion-control paradigm or confirmed publication novelty. Its value is that the successive research questions now have distinct mathematical objects and evidence boundaries. The account draws on the author’s March–October posts, September discussion records, standalone SCP code, proof files, and historical data. These original artifacts remain in their respective research workspaces and are not bundled with this self-contained manuscript; independent reproduction requires them.

REFERENCES

- [1] F. P. Kelly, A. K. Maulloo, and D. K. H. Tan, “Rate control for communication networks: Shadow prices, proportional fairness and stability,” *J. Operational Research Society*, vol. 49, pp. 237–252, 1998.
- [2] P. X. Gao et al., “pHost: Distributed near-optimal datacenter transport over commodity network fabric,” in *CoNEXT*, 2015. <https://conferences2.sigcomm.org/co-next/2015/img/papers/conext15-final1.pdf>
- [3] I. Cho, K. Jang, and D. Han, “Credit-scheduled delay-bounded congestion control for datacenters,” in *SIGCOMM*, 2017, pp. 239–252.
- [4] K. Prasopoulos et al., “SIRD: A sender-informed, receiver-driven datacenter transport protocol,” in *NSDI*, 2025. <https://www.usenix.org/conference/nsdi25/presentation/prasopoulos>
- [5] Google BBR Team, “BBR Startup pacing gain: A derivation,” technical note, June 2018. https://github.com/google/bbr/blob/master/Documentation/startup/gain/analysis/bbr_startup_gain.pdf
- [6] I. Swett and Google BBR Team, “BBR Startup cwnd gain: A derivation,” technical note, July 2018. https://github.com/google/bbr/blob/master/Documentation/startup/gain/analysis/bbr_startup_cwnd_gain.pdf
- [7] A. Aggarwal, S. Savage, and T. Anderson, “Understanding the performance of TCP pacing,” in *IEEE INFOCOM*, 2000, pp. 1157–1165.
- [8] M. Kempf et al., “QUIC steps: Evaluating pacing strategies in QUIC implementations,” *CoNEXT*, 2025; author version arXiv:2505.09222.